

SKILL Week 3

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

→ Creating a main loop

First check if src and all other file are available

```
varshini@DESKTOP-BLT5L3R:~$ cd ~/OSSP
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls
LICENSE  README.md  bin  fork_exec  fork_exec.c.save  obj  src
Makefile assets  docs fork_exec.c include  scripts tests
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls src
builtin.c executor.c main.c parser.c shell.c
```

So here I added a main loop in main.c in src

```
GNU nano 7.2 src/main.c
#include <stdio.h>
#include <string.h>

int main()
{
    while (1)
    {
        char input[1024];

        printf("OSSP> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nExiting shell...\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Now compile it

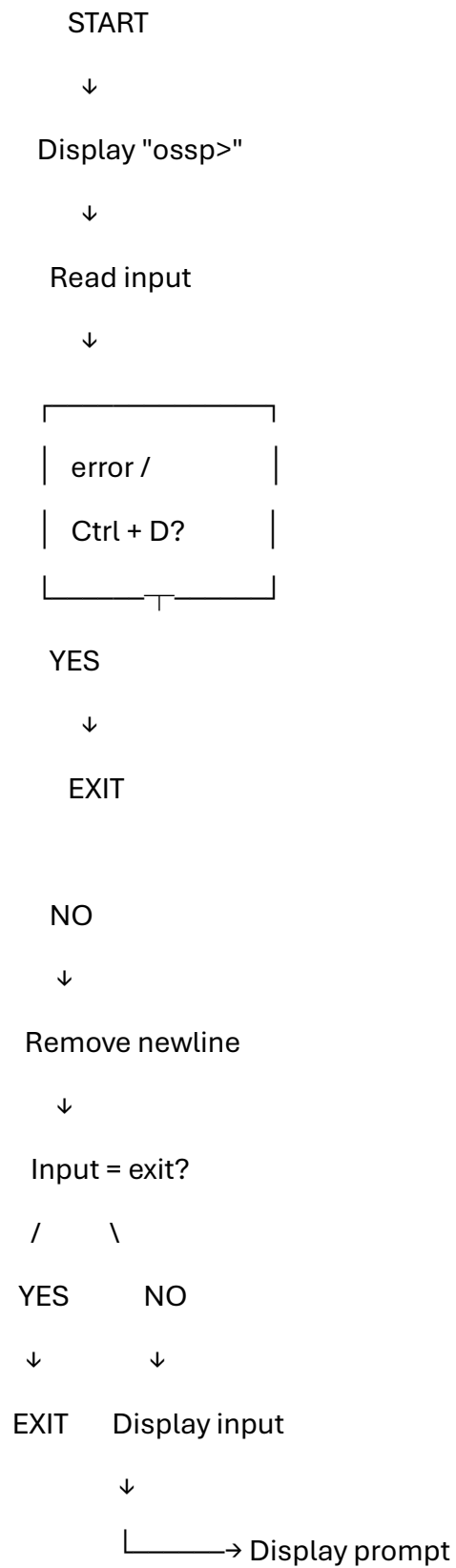
```
varshini@DESKTOP-BLT5L3R:~/OSSP$ nano src/main.c
varshini@DESKTOP-BLT5L3R:~/OSSP$ gcc src/main.c -o bin/shell
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls bin
my_shell  shell
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./bin/shell
osp> hello
You entered: hello
osp> test
You entered: test
osp> exit
Exiting shell...
```

By this we handled exit conditions

CONTROL FLOW

OSSP/

```
| ├── src/
|   └── main.c    ← Task 1 main loop
|
| ├── bin/
|   └── shell     ← compiled program
|
| ├── include/
| ├── obj/
| ├── docs/
| ├── tests/
| ├── scripts/
| ├── assets/
| ├── Makefile
| ├── README.md
| └── LICENSE
```



This part indicates testing of interactive loops

```
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./bin/shell
ossp> hello
You entered: hello
ossp> test
You entered: test
ossp> exit
Exiting shell...
```

Task 2

1) Open()

```
varshini@DESKTOP-BLT5L3R:~$ cd ~/OSSP
varshini@DESKTOP-BLT5L3R:~/OSSP$ pwd
/home/varshini/OSSP
varshini@DESKTOP-BLT5L3R:~/OSSP$ touch File1.txt
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls
File1.txt  README.md  docs          fork_exec.c.save  scripts
LICENSE   assets     fork_exec     include           src
Makefile  bin        fork_exec.c  obj              tests
varshini@DESKTOP-BLT5L3R:~/OSSP$ nano File1.txt
```

After creating file one

```
GNU nano 7.2 File1.txt
This is file1
This content will be copied to File2.
```

Create copy.c

```
varshini@DESKTOP-BLT5L3R:~/OSSP$ cat File1.txt
This is file1
This content will be copied to File2.
varshini@DESKTOP-BLT5L3R:~/OSSP$ nano copy.c
```

```
GNU nano 7.2 copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    printf("Files opened successfully\n");

    close(f1);
    close(f2);

    return 0;
}
```

```

varshini@DESKTOP-BLT5L3R:~/OSSP$ gcc copy.c -o copy
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls
File1.txt  README.md  copy      fork_exec      include  src
LICENSE    assets     copy.c    fork_exec.c    obj      tests
Makefile   bin        docs      fork_exec.c.save  scripts
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./copy
Files opened successfully
varshini@DESKTOP-BLT5L3R:~/OSSP$ ls
File1.txt  Makefile  bin      docs      fork_exec.c.save  scripts
File2.txt  README.md  copy     fork_exec  include          src
LICENSE    assets     copy.c   fork_exec.c  obj              tests

```

The files are opened successfully!

2. Read()

Now to read the file I have slightly edited copy.c

```

GNU nano 7.2                                copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;
    char buffer[100];

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    read(f1, buffer, 100);

    printf("%s", buffer);

    close(f1);
    close(f2);

    return 0;
}

```

```

varshini@DESKTOP-BLT5L3R:~/OSSP$ gcc copy.c -o copy
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./copy
This is file1
This content will be copied to File2.

```

So after editing copy.c giving the read function we have successfully read file1.

3. Write()

Again open copy.c add write element

```
GNU nano 7.2                                copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;
    char buffer[100];

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    read(f1, buffer, 100);

    printf("%s", buffer);

    write(f2, buffer, 100);

    close(f1);
    close(f2);

    return 0;
}
```

```
varshini@DESKTOP-BLT5L3R:~/OSSP$ nano copy.c
varshini@DESKTOP-BLT5L3R:~/OSSP$ gcc copy.c -o copy
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./copy
This is file1
This content will be copied to File2.
varshini@DESKTOP-BLT5L3R:~/OSSP$ cat File2.txt
This is file1
This content will be copied to File2.
```

After copying the text its time to close it

4. Close()

The close() in the program is nothing but closing files

```
GNU nano 7.2                                copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;
    char buffer[100];

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    read(f1, buffer, 100);

    printf("%s", buffer);

    write(f2, buffer, 100);

    close(f1);
    close(f2);

    return 0;
}
```

```
varshini@DESKTOP-BLT5L3R:~/OSSP$ nano copy.c
varshini@DESKTOP-BLT5L3R:~/OSSP$ gcc copy.c -o copy
varshini@DESKTOP-BLT5L3R:~/OSSP$ ./copy
This is file1
This content will be copied to File2.
varshini@DESKTOP-BLT5L3R:~/OSSP$ cat File1.txt
This is file1
This content will be copied to File2.
varshini@DESKTOP-BLT5L3R:~/OSSP$ cat File2.txt
This is file1
This content will be copied to File2.
varshini@DESKTOP-BLT5L3R:~/OSSP$ |
```

Therefore we finally completed closing the files and copying the text.

Task 3:

Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
varshini@DESKTOP-BLT5L3R:~$ cd ~/OSSP
varshini@DESKTOP-BLT5L3R:~/OSSP$ cd src
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ nano input.c
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ gcc input.c -o input
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./input
Enter a character: K

You entered: K
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ nano input.c
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ gcc input.c -o input
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./input
shell> hello

Command entered: hello
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./input
shell> ls

Command entered: ls
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./input
shell> pwd

Command entered: pwd
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ nano input.c
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ gcc input.c -o input
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./input
shell> hello
hello
Command entered: hello
shell> ls
ls
Command entered: ls
shell> pwd
pwd
Command entered: pwd
shell> exit
exit
Command entered: exit
Exiting shell...
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ |
```


Task 4:

```
GNU nano 7.2                                history.c
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

int main()
{
    char history[5][50];
    char input[50];
    char ch;
    int count = 0;
    int pos = 0;
    int h;

    struct termios old, new;

    tcgetattr(0, &old);
    new = old;
    new.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(0, TCSANOW, &new);

    while (1)
    {
        printf("\nShell> ");
        fflush(stdout);

        pos = 0;
        h = count;

        while (1)
        {
            read(0, &ch, 1);
```

```
        /* Enter */
        if (ch == '\n')
        {
            input[pos] = '\0';

            if (strcmp(input, "exit") == 0)
                break;

            strcpy(history[count], input);

            if (count < 4)
                count++;

            break;
        }

        /* Arrow keys */
        if (ch == 27)
        {
            read(0, &ch, 1);
            read(0, &ch, 1);

            /* UP */
            if (ch == 'A')
            {
                if (h > 0)
                {
                    h--;
                    printf("\rShell> %s", history[h]);
                    strcpy(input, history[h]);
                    pos = strlen(input);
                }
            }
        }
    }
}
```

```

        /* DOWN */
        else if (ch == 'B')
        {
            if (h < count - 1)
            {
                h++;
                printf("\rShell> %s", history[h]);
                strcpy(input, history[h]);
                pos = strlen(input);
            }
        }

        /* Normal character */
        else
        {
            input[pos] = ch;
            pos++;
            printf("%c", ch);
        }

        fflush(stdout);
    }

    if (strcmp(input, "exit") == 0)
        break;
}

tcsetattr(0, TCSANOW, &old);

return 0;
}

```

Now press ↑.

You should see the previous command.

```

varshini@DESKTOP-BLT5L3R:~$ cd ~/OSSP/src
nano history.c
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ gcc history.c -o history
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./history

Shell> hello
Shell> ls
Shell> pwd
Shell> hello

```

Task 5

```
GNU nano 7.2                                memory.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    /* 1. Dynamic buffer allocation */
    int size = 5;
    char *buffer = malloc(size * sizeof(char));

    if (buffer == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter a string (maximum %d characters): ", size - 1);
    fgets(buffer, size, stdin);

    printf("Buffer contains: %s", buffer);

    /* 2. Resize the buffer */
    size = 20;

    char *temp = realloc(buffer, size * sizeof(char));

    if (temp == NULL) {
        printf("Memory reallocation failed\n");
        free(buffer);
        return 1;
    }

    buffer = temp;

    printf("\nEnter another string (maximum %d characters): ", size - 1);
    fgets(buffer, size, stdin);

    printf("Resized buffer contains: %s", buffer);

    /* 3. Linked list */
    struct Node *head = malloc(sizeof(struct Node));
```

```

    if (head == NULL) {
        free(buffer);
        return 1;
    }

    head->data = 100;
    head->next = NULL;

    struct Node *second = malloc(sizeof(struct Node));

    if (second == NULL) {
        free(head);
        free(buffer);
        return 1;
    }

    second->data = 200;
    second->next = NULL;

    head->next = second;

    printf("\nLinked List:\n");

    struct Node *current = head;

    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }

    printf("NULL\n");

    /* 4. Release memory */
    free(second);
    free(head);
    free(buffer);

    printf("\nMemory released successfully.\n");

    return 0;
}

```

```

varshini@DESKTOP-BLT5L3R:~/OSSP/src$ cd ~/OSSP/src
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ nano memory.c
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ gcc memory.c -o memory
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ ./memory
Enter a string (maximum 4 characters): hello
Buffer contains: hell
Enter another string (maximum 19 characters): Resized buffer contains: o

Linked List:
100 -> 200 -> NULL

Memory released successfully.

```

Valgrind Installation

```
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ sudo apt update
sudo apt install valgrind
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
82 packages can be upgraded. Run 'apt list --upgradable' to see them.
E: dpkg was interrupted, you must manually run 'sudo dpkg --configure -a' to
correct the problem.
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ sudo dpkg --configure -a
Setting up plocate (1.1.19-2ubuntu2) ...
Created symlink /etc/systemd/system/timers.target.wants/plocate-updatedb.tim
er → /usr/lib/systemd/system/plocate-updatedb.timer.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ valgrind --leak-check=full ./memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gdb libbabeltrace1 libc6-dbg libdebuginfod-common libdebuginfod1t64
  libipt2 libsource-highlight-common libsource-highlight4t64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind alleyoop
  valkyrie
```

```
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ valgrind --version
valgrind-3.22.0
varshini@DESKTOP-BLT5L3R:~/OSSP/src$ valgrind --leak-check=full ./memory
==1662== Memcheck, a memory error detector
==1662== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1662== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1662== Command: ./memory
==1662==
Enter a string (maximum 4 characters): abc
Buffer contains: abc

Enter another string (maximum 19 characters): hello world
Resized buffer contains: hello world

Linked List:
100 -> 200 -> NULL

Memory released successfully.
==1662==
==1662== HEAP SUMMARY:
==1662==    in use at exit: 0 bytes in 0 blocks
==1662==   total heap usage: 6 allocs, 6 frees, 2,105 bytes allocated
==1662==
==1662== All heap blocks were freed -- no leaks are possible
==1662==
==1662== For lists of detected and suppressed errors, rerun with: -s
==1662== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
varshini@DESKTOP-BLT5L3R:~/OSSP/src$
```

